



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

Chapter 2

Java Fundamentals



Chapter 2

- Identifiers, keywords, separators, comments
- Basic Java data types and constants
- Control process
- Inputs and outputs
- Array
- Exception mechanisms
- Java Virtual Machine and garbage collection



Identifier

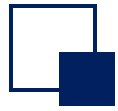
❑ **Identifier:** A valid character series used to identify class names, variable names, method names, object names, array names, and file names.

❑ **Specification of identifiers:**

The identifier can be a combination of letters, numbers, underscores, and dollar sign \$, and the length is unlimited.

The identifier must start with an alphabet, an underscore, and dollar sign \$, and **cannot start with a number.**

Java identifiers are case-sensitive.



Identifier

NO.1

Capitalize the first letter of each word of the **class name**, such as: **HelloWorldClass**.

NO.4

Constant names are all capitalized, separated by underscores. As: **MONTH_OF_YEAR**.

Java naming conventions

NO.2

Identifiers such as **variable names, object names, method names, and package names** are all in lowercase letters. If the identifier consists of more than one word, its first letter is lowercase, and all words after it should capitalize the first letter, e.g. getAge.

NO.3

It cannot have the same name as a keyword or a special value, such as: **false, for, while**



Identifier

// Class names

```
public class Student { }  
public class StudentManager { }  
public class CourseEnrollment { }
```

// Variable names

```
String studentName;  
int studentId;  
double gpa;  
boolean isEnrolled;
```

// Method names

```
public void enrollStudent() { }  
public double calculateGPA() { }  
public boolean isEligibleForGraduation() { }
```

// Object names

```
Student newStudent = new Student();  
Course mathCourse = new Course();  
StudentManager manager = new StudentManager();
```

// Array names

```
String[] studentNames;  
int[] studentGrades;  
Student[] enrolledStudents;
```

// Constants

```
public static final int MAX_STUDENTS = 1000;  
public static final double MINIMUM_GPA = 2.0;  
public static final int MAX_COURSES_PER_SEMESTER = 6;
```

// Package names

```
package com.university.studentmanagement;  
package com.university.course.enrollment;  
package com.university.grade.calculation;
```



Keywords

Keywords In Java

- | | | | |
|--------------|----------------|---------------|------------------|
| 1. abstract | 13. double | 25. int | 37. strictfp |
| 2. assert | 14. else | 26. interface | 38. super |
| 3. boolean | 15. enum | 27. long | 39. switch |
| 4. break | 16. extends | 28. native | 40. synchronized |
| 5. byte | 17. final | 29. new | 41. this |
| 6. case | 18. finally | 30. package | 42. throw |
| 7. catch | 19. float | 31. private | 43. throws |
| 8. char | 20. for | 32. protected | 44. transient |
| 9. class | 21. if | 33. public | 45. try |
| 10. continue | 22. implements | 34. return | 46. void |
| 11. default | 23. import | 35. short | 47. volatile |
| 12. do | 24. instanceof | 36. static | 48. while |



Keywords

❑ Data Type Keywords

- int, double, boolean, char, float, long, short, byte

❑ Control Flow Keywords

- if, else, while, for, switch, case, break, continue

❑ Class and Method Keywords

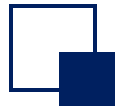
- class, public, private, protected, static, void, return

❑ Object Creation Keywords

- new, this, super, extends, implements

❑ Exception Handling Keywords

- try, catch, finally, throw, throws



separator

semicolon ;

The end of the
identification
statement cannot be
omitted.

For example:

```
{x=12;y=4;}
```

braces { }

Indicate the scope of
the class, the scope of
the method body, the
scope of the compound
statement, and the
initialization of array
members. as:

```
if condition {...}  
else {...}
```

bracket (), []

() flags used for cast
castings or used as
function headers; [] is
used for the definition
of arrays and
references to elements。

For example:

```
int[] score = new int[30];
```




separator

```
public class Student {           // Opening brace for class
    private String name;         // Code inside class

    public void study() {        // Opening brace for method
        System.out.println("Studying..."); // Code inside method
    }                            // Closing brace for method
}                                // Closing brace for class
```

```
// Array declaration
int[] studentGrades;           // Square brackets declare array

// Array creation
studentGrades = new int[5];     // Square brackets specify size

// Array initialization
int[] grades = {85, 90, 78, 92, 88}; // Square brackets for initialization

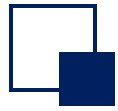
// Accessing array elements
int firstGrade = grades[0];     // Square brackets access element
grades[1] = 95;                 // Square brackets modify element
```

```
// Method definition with parameters
public Student(String name, int id) { // Parentheses contain parameters
    this.name = name;
    this.studentId = id;
}

// Method call
student.displayInfo();           // Parentheses for method call

// Expression grouping
double average = (grade1 + grade2 + grade3) / 3; // Parentheses group math

// Type casting
int age = (int) studentGPA;     // Parentheses for casting
```



Comments

Line Comments

//

```
// Add below  
int a = 1 + 2;
```

Block Comments

/* */

```
/* Program Name:  
   Function:  
   Version:  
   Writing time: */
```

Document Comments

/** */

```
/**  
    Document  
    annotation content  
    */  
The content of the  
document comments  
can be automatically  
generated by using the  
javadoc command.
```



Comments

```
public class Student {  
    /*  
     * This class represents a student in our management system.  
     * It stores basic information like name, ID, and GPA.  
     * Created by: Prof. Smith  
     * Date: 2024  
     * Version: 1.0  
     */  
  
    private String name;  
    private int studentId;  
  
    public void calculateGPA() {  
        /*  
         * This method calculates the student's GPA based on their grades.  
         * It takes all grades, finds the average, and updates the gpa field.  
         */  
        // Implementation code here  
    }  
  
    /* Temporarily disabled for testing  
    public void sendEmail() {  
        // Email sending code  
    }  
    */  
}
```



Comments

```
/**
 * Represents a student in the university management system.
 * This class stores student information and provides methods
 * to manage student data.
 *
 * @author Prof. Smith
 * @version 1.0
 * @since 2024-01-01
 */
public class Student {
    private String name;
    private int studentId;
    private double gpa;

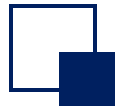
    /**
     * Creates a new student with the given name and ID.
     *
     * @param name the student's full name
     * @param studentId the unique student identifier
     */
    public Student(String name, int studentId) {
        this.name = name;
        this.studentId = studentId;
    }

    /**
     * Calculates and returns the student's GPA.
     *
     * @return the student's current GPA
     * @throws IllegalStateException if no grades have been recorded
     */
    public double getGPA() {
        if (gpa == 0.0) {
            throw new IllegalStateException("No grades recorded yet");
        }
        return gpa;
    }
}
```



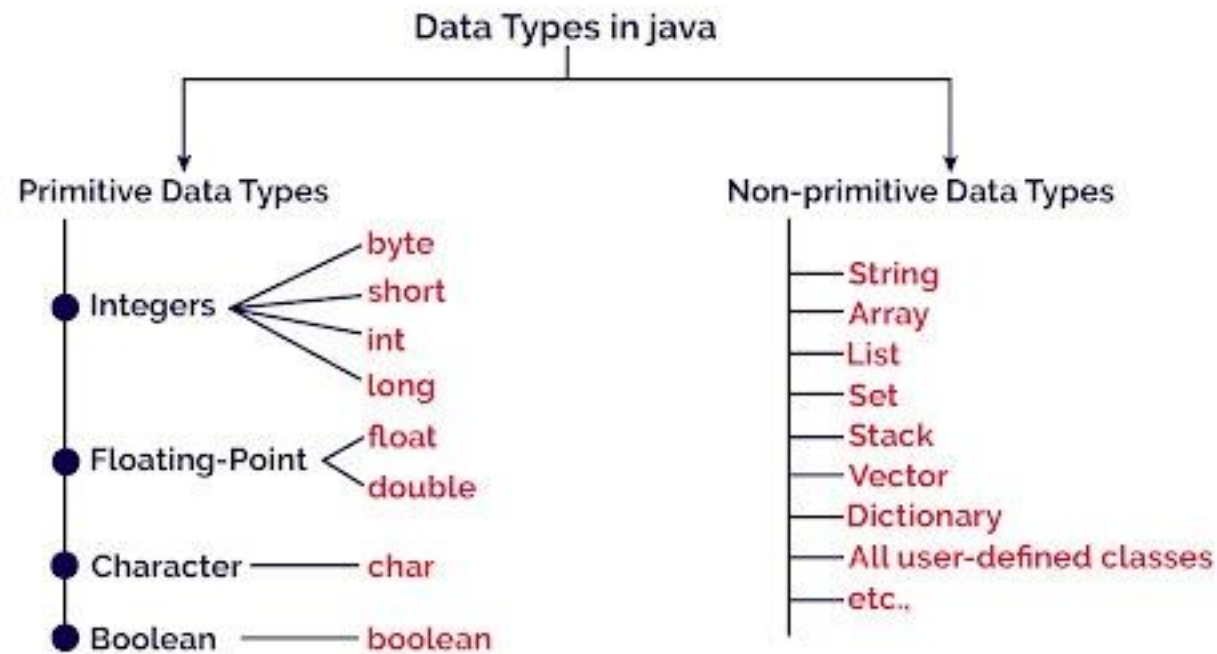
Chapter 2

- Identifiers, keywords, separators, comments
- **Basic Java data types and constants**
- Control process
- Inputs and outputs
- Array
- Exception mechanisms
- Java Virtual Machine and garbage collection



Java basic data types

- ❑ Java divides variables into two categories based on data types: primitive data types and non-primitive data types
- ❑ There are eight primitive data types: byte, short, int, long, float, double, char, and boolean



Integer Type: byte、 short、 int、 long

- ❑ Each integer type in Java has a fixed range of tables and field lengths, which are not affected by the specific operating system, so as to ensure the portability of Java programs
- ❑ Java's shaping constants default to int, declaring long constants must be accompanied by 'l' or 'L' at the end of the number

type	storage space	The range of numbers
byte	1byte=8bit	-128 ~ 127
short	2byte	$-2^{15} \sim 2^{15}-1$
int	4byte	$-2^{31} \sim 2^{31}-1$ (About 2 billion)
long	8byte	$-2^{63} \sim 2^{63}-1$

Floating-point type: float、 double

- ❑ The floating-point type of Java adopts the IEEE floating-point format, which has a fixed range of tables and field lengths, and is not affected by the specific operating system.
- ❑ Java's floating-point constants default to double, and float-type constants must be declared with 'f' or 'F' at the end of the number.

type	Takes up storage space	The range of table numbers
Single-precision float	4byte	-3.403E38 ~ 3.403E38
Double-precision double	8byte	-1.798E308 ~ 1.798E308

Floating-point type: float、double

❑ The floating-point type constructor:

```
float f = 3.14f  
float f = new Float(3.14)
```

```
float f = 3.14
```



❑ Type Promotion :

```
public class Main {  
    public static void main(String[] args) {  
        int n = 5;  
        double d = (1.2 + 24.0)/n;  
        System.out.println(d); //5.04  
    }  
}
```

```
double d = 1.2 + 24 / 5;
```

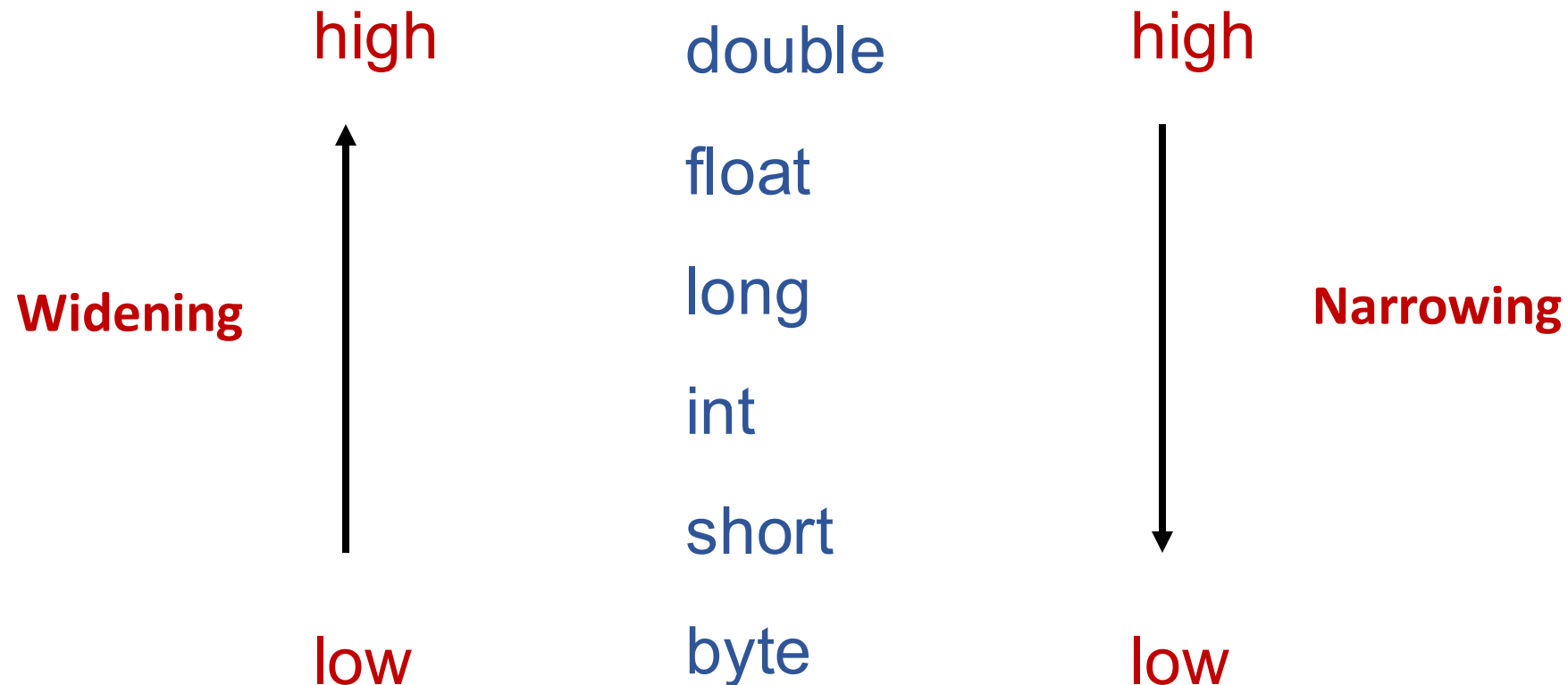
```
// 5.2
```

?



Conversion between different types of data

- A conversion of a data type occurs when the value of one data type is assigned to a variable of another data type.
- In the integer type and the floating-point number, you can arrange the data types in order of precision from "high" to "low" as follows:



Type conversion rules

- ❑ When a low-level value is assigned to a high-level variable, the system automatically completes the conversion of the data type.

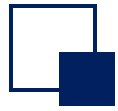
Example: `float x=200;` Convert the int value of type 200 to the value of float type 200.0

- ❑ When a high-level value is assigned to a low-level variable, a cast conversion must be performed.
 - Forced syntax format: (target type name) the value to be converted; The value can be an expression

Example: `int i;`

`i = (int) 26L;` Convert the value of type long 26 to the value of type int 26.

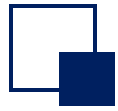
- **When you perform a cast conversion, you may lose data precision.**



Type conversion rules

Widening

```
public class WideningConversion {  
    public void demonstrateWidening() {  
        // byte → short → int → long  
        byte studentAge = 20;  
        short ageInMonths = studentAge;    // Automatic: 20  
        int totalAge = ageInMonths;        // Automatic: 20  
        long bigAge = totalAge;            // Automatic: 20  
  
        // int → float → double  
        int studentId = 1001;  
        float floatId = studentId;         // Automatic: 1001.0  
        double doubleId = floatId;        // Automatic: 1001.0  
  
        // char → int  
        char grade = 'A';                  // ASCII 65  
        int gradeValue = grade;            // Automatic: 65  
    }  
}
```



Type conversion rules

Narrowing

```
public class NarrowingConversion {  
    public void demonstrateNarrowing() {  
        // double → float  
        double preciseGPA = 3.789123456789;  
        float approximateGPA = (float) preciseGPA; // 3.7891235 (precision loss)  
  
        // int → short  
        int largeCount = 40000; // 40000 > short.MAX_VALUE (32767)  
        short shortCount = (short) largeCount; // Overflows to -25536  
  
        // int → byte  
        int largeAge = 200; // 200 > byte.MAX_VALUE (127)  
        byte byteAge = (byte) largeAge; // Overflows to -56  
  
        // double → int  
        double gpa = 3.75;  
        int intGPA = (int) gpa; // 3 (decimal part lost)  
    }  
}
```

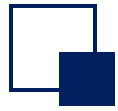
String numbers are converted to numeric values

❑ Example of converting to an integer:

```
int x1 = Integer.parseInt("260"); //String direct conversion, x1=260  
int x2 = Integer.parseInt(txt1.getText()); //Convert the text in the text box txt1 to int and assign it to x2
```

❑ Example of converting to floating-point:

```
float y = Float.parseFloat("23.5");  
double z = Double.parseDouble("45.6");
```



Character Type: char

- ❑ The Java language specification states that the char type of Java is a UTF-16 code unit, which is 16 bits (2 bytes)

- ❑ There are three types of character-based variables:

- A character variable is a single character enclosed in single quotation marks. For example:

```
char c1 = 'a'; char c2 = '中' ; char c3 = '9';
```

- Java also allows the use of the escape character '\\' to convert subsequent characters into special character constants. For example:

```
char c3 = '\n';
```

Indicates a line break

- Use the Unicode value directly to represent the character constant: '\uXXXX'. where XXXX represents a hexadecimal integer. For example, \u000a means \n.

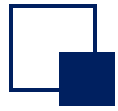
- ❑ The char type can be computed because it corresponds to Unicode code.



Boolean type: boolean

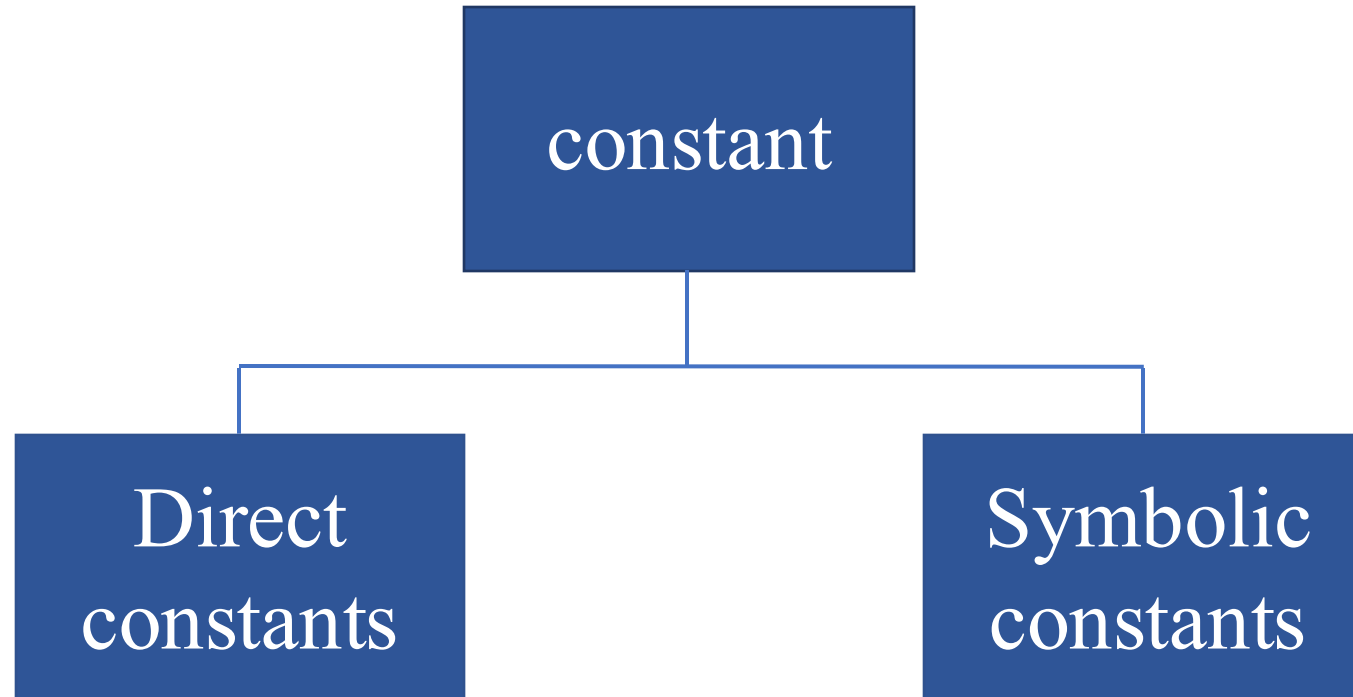
- ❑ The boolean type is used to determine the logical conditions, and is generally used for program flow control:
 - If Conditional control statements; while Loop control statements; do-while Loop control statements; for Loop control statements.
 - **Boolean can only be true or false**
 - You can't use 0 or non-0 integers instead of false and true, unlike in C.

```
boolean b1 = true  
boolean b2 = false
```

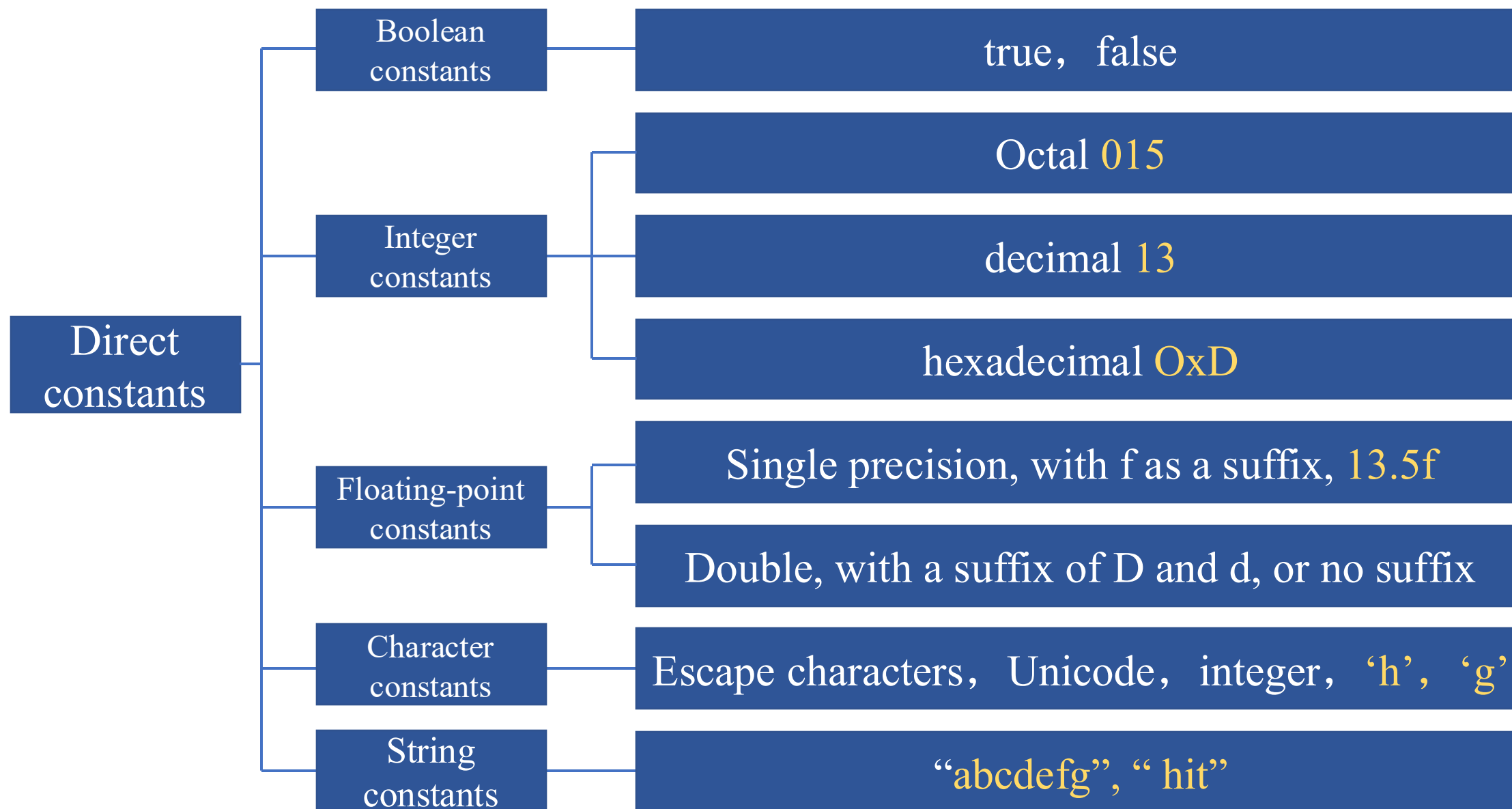
Constants and variables

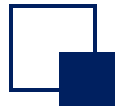
- **Constant**: A quantity whose value remains constant during the running of a program.





Constants and variables





Constants and variables

- **Symbolic constants:** Constants represented by identifiers should be declared before they are used.

[modifier] final type identifier constant name = (direct) constant;

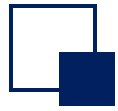
- A modifier is a permission modifier that indicates the scope of application of the constant, and the modifiers that can be adopted are:

public, private, protected or default

- **final** is a keyword that defines a constant
- There are three types of constants (with different scopes): static, member, and local

```
public final static Color black = new Color(0, 0, 0);  
public final static Color BLACK = black;  
public final static double PI = 3.1415926535;
```

1. Initialization is required at the time of definition
2. Constants are generally uppercase characters



Constants and variables

- **Variable: refers to the amount of value that can be changed during the running of the program.**
Java stipulates that variables must be defined before they are used.

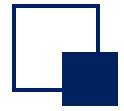
[Modifier] Type_Identifier Variable_Name [=Constant];

```
float x = 25.4, y;  
char c;  
boolean flag1 = true;
```



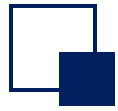
Chapter 2

- Identifiers, keywords, separators, comments
- Basic Java data types and constants
- **Control process**
- Inputs and outputs
- array
- Exception mechanisms
- Java Virtual Machine and garbage collection



Java control process

- **Conditional statements**
- **Loop statements**
- **Jump statement**



Conditional statements

□ If

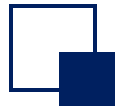
With the if conditional, you can choose whether you want to execute the statement immediately after the condition

```
int a = 100;
if(a == 100)
{
    System.out.println(a);
}
```

□ if...else

If a certain condition is met, one treatment is carried out, otherwise another treatment is carried out

```
if(math >= 60) { // 判断条件: math大于等于60成立
    System.out.println("math has passed");
} else { // 判断条件不满足
    System.out.println("math has not passed");
}
```



Conditional statements

□if...else if

If a certain condition is met, a certain treatment is carried out,

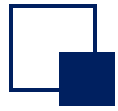
Otherwise, if another condition is met, another treatment is carried out.

□Switch Multi-branching statements

The value of the expression in the switch statement must be an integer, a character, or a string, and the constant value 1~n must also be an integer, character, or string.

```
if(x > 60) {  
    System.out.println("x is greater than 60");  
} else if (x > 30) {  
    System.out.println("x is greater than 30 but less than  
or equal to 60");  
} else if (x > 0) {  
    System.out.println("x is greater than 0 but less than  
or equal to 30");  
} else {  
    System.out.println("x is less than or equal to 0");  
}
```

```
switch (expression) {  
    case Constant value 1:  
        Statement block 1  
        [break;]  
  
    ...  
    case Constant value n:  
        Statement block n  
        [break;]  
  
    default:  
        Statement block n+1;  
        [break;]  
}
```

Loop statements

□while

The while loop statement is looped in such a way that a condition is used to control whether the statement should continue to be executed repeatedly.

```
while (x <= 10) {  
    sum += x;  
    x++;  
}
```

□do...while

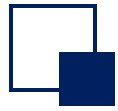
The difference between a **do...while** loop and a **while** loop statement is that the **while** loop statement first determines whether the condition is true and then executes the loop body, while the **do...while** loop is executed once before determining whether the condition is true. That is, **do...while** will be performed at least once.

```
do {  
    System.out.println("b == " + b);  
    b--;  
}while(b >= 8)
```

□for

A for loop can be used to repeat a statement until expression 2 is satisfied

```
for(Expression 1; Expression 2;  
Expression 3)  
{  
    Statement sequences  
}
```



Jump statement

The Java language provides three types of jump statements: break statement, continue statement, and return statement.

❑ break

The break statement can force exit the current loop. Suppose two loops are nested and break is used under the inner loop, then break can only jump out of the inner loop.

❑ continue

The continue statement is used to allow the program to skip the statement that follows it and proceed to the next loop.

❑ return

A return statement can return from a method and give control to the statement that called it.

```
for(int i=0; i<n; i++) {    // Outer Loop
    for(int j=0; j<n; j++) {    // inner Loop
        break;
    }
}
```

```
int i = 0;
while (i < 10) {
    i++;
    if(i % 2 == 0) {
        continue;    // If it's an even number,
        skip the current loop
    }
    System.out.println(i);
}
```

```
public void getName() {
    return name;
}
```



Classroom exercises

Please give the result of 4 pieces of code

```
int i = 0;
while (i < 10) {
    i++;
    if(i % 2 == 0) {
        continue; // If it's an even number,
        skip the current loop
    }
    System.out.println(i);
}
```

Results: 1, 3, 5, 7, 9

```
int i = 0;
while (i < 10) {
    if(i % 2 == 0) {
        continue; // If it's an even number,
        skip the current loop
    }
    System.out.println(i);
    i++;
}
```

Result: No output

```
int i = 0;
while (i < 10) {
    System.out.println(i);
    i++;
    if(i % 2 == 0) {
        continue; // If it's an even number,
        skip the current loop
    }
}
```

Results: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

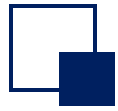
```
int i = 0;
while (i < 10) {
    i++;
    if(i % 2 == 0) {
        break; // If it's an even number
        skip loop
    }
    System.out.println(i);
}
```

Results:1



Chapter 2

- Identifiers, keywords, separators, comments
- Basic Java data types and constants
- Control process
- **Inputs and outputs**
- Array
- Exception mechanisms
- Java Virtual Machine and garbage collection



Java input and output

□ Standard input: System.in

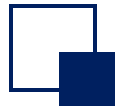
System.in is an object of the InputStream class that is not typically used directly to read input from the user's keyboard. Instead, two commonly used packages are available:

1. Encapsulate the System.in with a stream of characters

```
BufferedReader stdin = new BufferedReader(new InputStreamReader(System.in));  
System.out.print("enter a Line: ")  
System.out.println(stdin.readLine());
```

2. Use java.util.Scanner to encapsulate the System.in

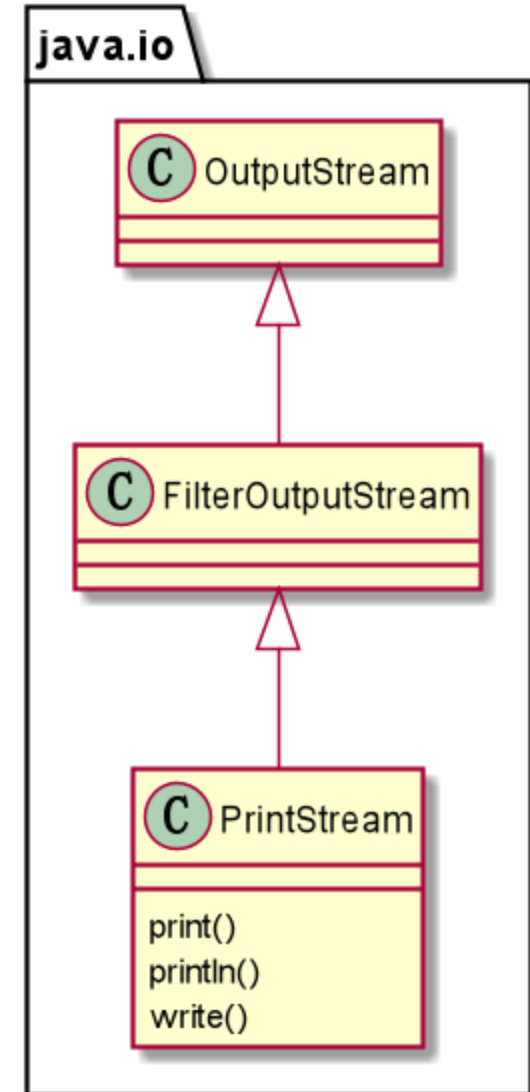
```
Scanner stdin = new Scanner(System.in);  
System.out.print("enter a Line: ");  
System.out.println(stdin.nextLine()); // Read a line of input and get  
a string
```

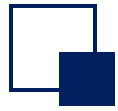


Java input and output

❑ Standard output: System.out

- System.out is an object of the PrintStream class that can be called to print, println, or write member methods to output various types of data in the console.
- The parameters of print and println are exactly the same, except that println is output and print does not wrap a line. The write method is used to output an array of bytes, which does not wrap lines.





Java formatting

Common Format Specifiers

Specifier	Description	Example
%s	String	"Hello"
%d	Integer	123
%f	Floating point	45.67
%n	New line	
%b	Boolean	true/false
%x	Hexadecimal	ff

Formatting Options

Option	Example	Result
Width	%10s	" Hello"
Left justify	%-10s	"Hello "
Precision	%.2f	3.14
Zero padding	%05d	00123
Thousands separator	%,d	1,000,000



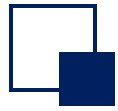
Java formatting

```
public void displayStudentList(ArrayList<Student> students) {  
    if (students.isEmpty()) {  
        System.out.println("No students found.");  
        return;  
    }  
  
    System.out.println("\nStudent List:");  
    System.out.println("-".repeat(60));  
    System.out.printf("%-5s %-20s %-10s %-10s\n", "ID", "Name", "GPA", "Status");  
    System.out.println("-".repeat(60));  
  
    for (Student student : students) {  
        String status = getStudentStatus(student.getGPA());  
        System.out.printf("%-5d %-20s %-10.2f %-10s\n",  
            student.getId(),  
            student.getName(),  
            student.getGPA(),  
            status);  
    }  
    System.out.println("-".repeat(60));  
}
```




Chapter 2

- Identifiers, keywords, separators, comments
- Basic Java data types and constants
- Control process
- Inputs and outputs
- **Array**
- Exception mechanisms
- Java Virtual Machine and garbage collection



Java arrays

The types on the right cannot be found in the Java standard library, but the Java Virtual Machine automatically creates array types, which can be treated as built-in types in Java like the 8 basic data types. The naming convention for this type goes like this:

- Each dimension is denoted by a '['; The first two '[' represent two-dimensional arrays
- '[' is followed by the type of the element in the array (including basic data types and reference data types)

```
int[] a = new int[5];  
System.out.println(a.getClass().getName());
```



[I

```
char[] b = new char[5];  
System.out.println(b.getClass().getName());
```



[C

```
String[] c = new String[5];  
System.out.println(c.getClass().getName());
```

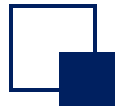


[Ljava.Long.String

```
String[][] d = new String[5];  
System.out.println(d.getClass().getName());
```



[[Ljava.Long.String



Java arrays

❑ Declare an array

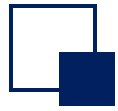
There are two ways to declare arrays, the first is the way the Java language specification promotes, and the second is the C-style way of declaring arrays.

```
int[] a;  
int b[];
```

❑ Create an array

There are two ways to create an array, the first one specifies the length of the array, and the second directly assigns a value to the array. Once an array is created, its length can no longer be changed.

```
int[] c = new int[2];  
int[] d = new int[]{0,1};
```



Java arrays

Access the array

The subscript of the array starts from 0 and goes all the way up to the length -1. An array is a randomly accessed data structure.

```
int[] a = new int[2];  
for(int i = 0; i < 2; i++){  
    a[i] = i;  
    System.out.println(a[i]);  
}
```

Array length

The length property is used to access the length of an array

```
int[] a = new int[2];  
System.out.println(a.length);
```

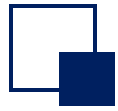
The array is out of bounds

The array access subscript range is 0 to length -1, and once this range is exceeded, an array subscript out-of-bounds exception will be generated

```
int[] a = new int[2];  
System.out.println(a[2]);
```



Java.Lang.ArrayIndexOutOfBoundsException: 2



Java collections out of bounds

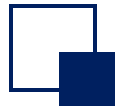
Access to a collection checks for out-of-bounds, and throws an out-of-bounds exception if it does, similar to arrays. Let's take a look at what Java's exception mechanism is:

```
private void rangeCheck(int index) {  
    if (index >= size)  
        throw new IndexOutOfBoundsException(outOfBoundsException(index));  
}
```

```
List<Integer> list = new ArrayList<>(initialCapacity: 2);  
list.add(1);  
list.add(2);  
System.out.println(list.get(2));
```



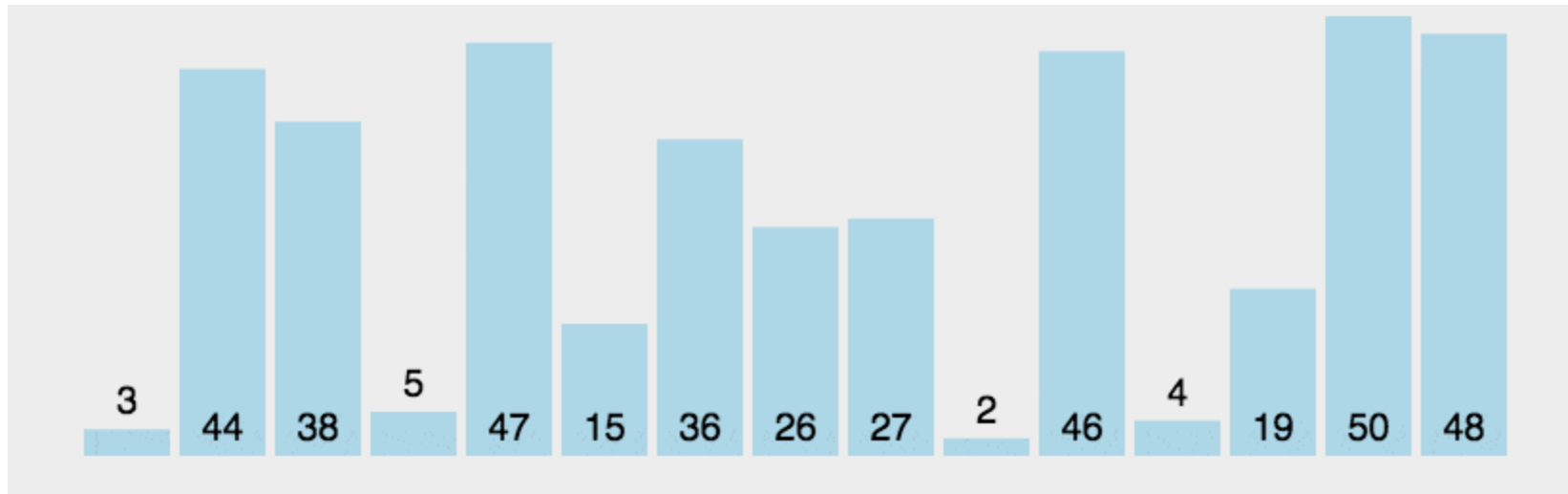
java.lang.IndexOutOfBoundsException: Index: 2, Size: 2



Classroom exercises

- Get hands-on with bubbling sorting. The input is an int array.
- Bubbling Sort Tips:

Starting with the first element, compare each element to the element to its right, and swap their positions if it is larger than the element on the right. After this step is done, the final element will be the largest number.





Classroom exercises

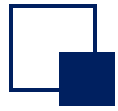
- Get hands-on with bubbling sorting. The input is an int array.

```
public static void sort(int[] a) {  
    //The outer loop controls the number of comparisons  
    for (int i = 0; i < a.length - 1; i++) {  
        //The inner layer loops to control the arrival position  
        for (int j = 0; j < a.length - i - 1; j++) {  
            //The elements in the front are swapped if they are  
            //larger than those in the back  
            if (a[j] > a[j + 1]) {  
                int temp = a[j];  
                a[j] = a[j + 1];  
                a[j + 1] = temp;  
            }  
        }  
    }  
}
```



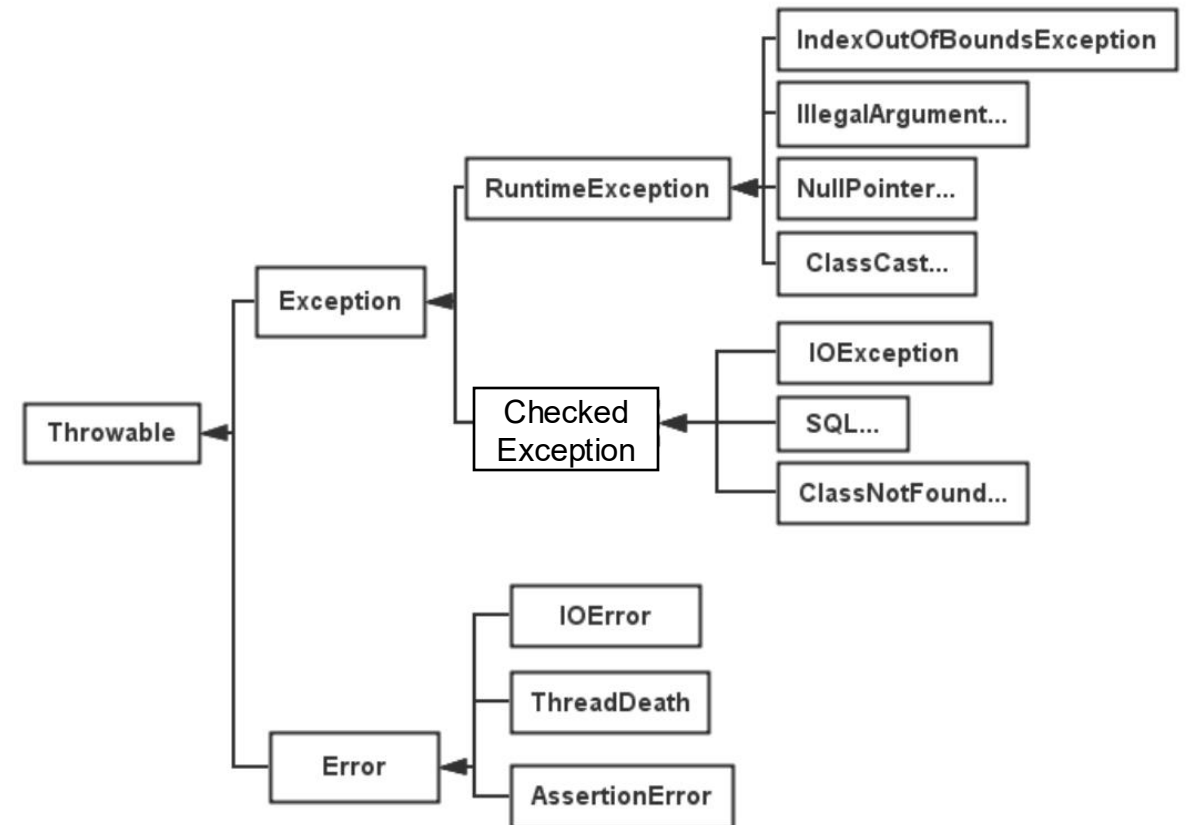
Chapter 2

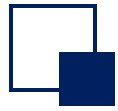
- Identifiers, keywords, separators, comments
- Basic Java data types and constants
- Control process
- Inputs and outputs
- Array
- **Exception mechanisms**
- Java Virtual Machine and garbage collection



Java exception mechanism

- ❑ Java exception is a consistent mechanism provided by Java to identify and respond to errors, which can separate the abnormal code from the normal business code in the program, ensure that the program code is more elegant, and improve the robustness of the program.
- ❑ Exceptions refer to unexpected conditions, such as file not found, network connection failure, and invalid parameters. An exception is an event that occurs during the running of a program and interferes with the normal flow of instructions. Java describes a variety of different exceptions through numerous subclasses of the Throwable class in the API.

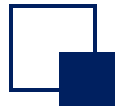




Java exception keyword

❑ Exception keywords:

- **try** – Used for listening. Put the code to be listened to (code that may throw an exception) in the try block, and when an exception occurs in the try block, the exception will be thrown.
- **catch** – Used to catch exceptions. Catch is used to catch exceptions that occur in the try block.
- **finally** – Finally blocks will always be executed. It is mainly used to reclaim resources (such as database connections, network connections, and disk files) that are opened in the try block. Only after the final block is executed, the return or throw statement in the try or catch block will be executed. If a statement with a termination method such as return or throw is used in finally, the execution will not be jumped back and will be stopped.
- **throws** – Used in method signatures to declare exceptions that may be thrown by the method.
- **throw** – Used to throw exceptions.



Throw and throws

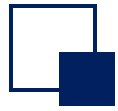
- **Common:**

Both are passive ways of handling exceptions, only responsible for throwing exceptions, but not handled by the function, the real handling of exceptions is handled by the upper layer of the function.

- **distinguish:**

- throws are used in the method header and represent only the declaration of the exception, while throw is used inside the method and throws the exception object.
- throws can throw multiple exceptions at once, while throw can only be one.

```
public static int getElement(int[] arr, int index) throws Exception, IOException {  
    if(arr==null) {  
        throw new NullPointerException("The pointer is empty");  
    }else if(index>=arr.length-1) {  
        throw new ArrayIndexOutOfBoundsException("The array is out of bounds");  
    }  
    int ele = arr[index];  
    return ele;  
}
```



Java exception mechanism

❑ Exceptions are usually caught and handled in the following ways:

- try-catch

Multiple exception types can be caught in a single try-catch block, and different types of exceptions can be handled differently

- try-finally

try block, the statement after the abnormal code is no longer executed, and the final statement is executed. If the try block does not throw an exception, the FINALLY statement is executed after the try block is executed.

- try-catch-finally

```
try{  
    // Execute the program code, an exception  
    may occur  
} catch(Exception1 e1) {  
    // Exception1 is caught and handled  
} catch(Exception2 e2) {  
    //Exception1 is caught and handled  
}  
...
```

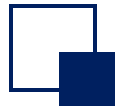
```
try{  
    // Execute the program code, an exception  
    may occur  
} finally {  
    // Code that will definitely be executed  
...  
}
```

```
try{  
    // Execute the program code, an exception  
    may occur  
} catch(Exception e) {  
    // Exceptions are caught and handled  
} finally {  
    // Code that will definitely be executed  
}  
...
```



Chapter 2

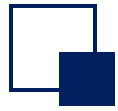
- Identifiers, keywords, separators, comments
- Basic Java data types and constants
- Control process
- Inputs and outputs
- array
- Anomalous mechanisms
- **Java Virtual Machine and garbage collection**



Java Virtual Machine

❑ JVM (Java Virtual Machine)

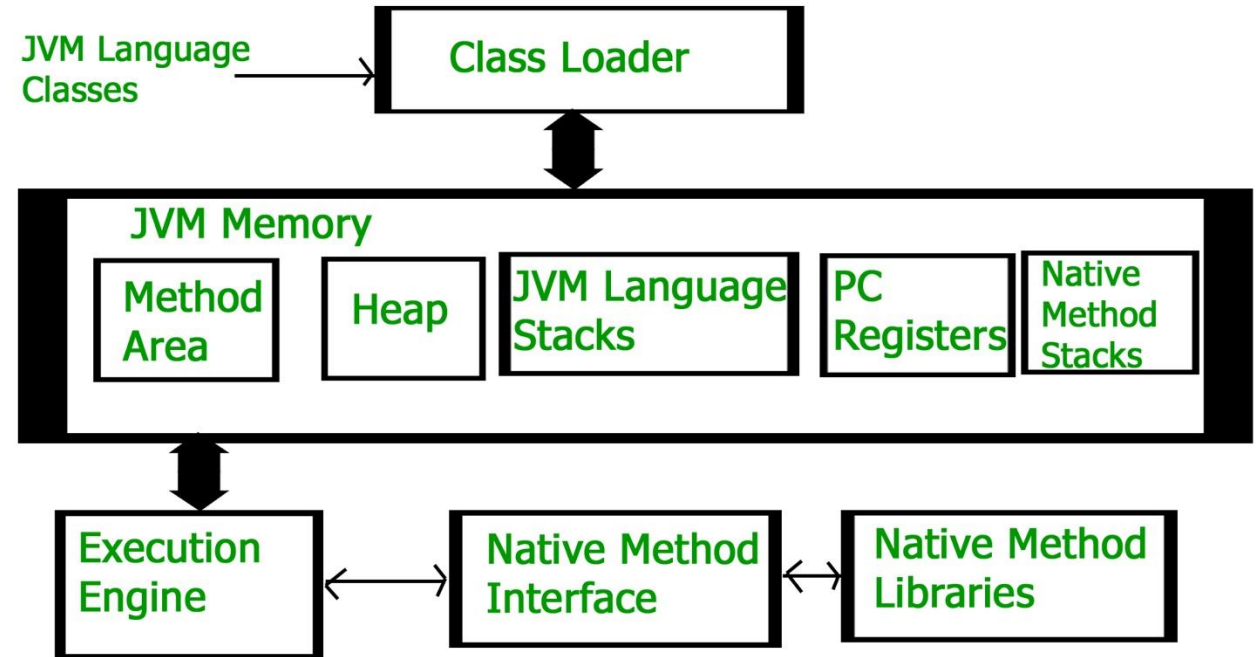
- **A virtual machine is a computer system with a complete hardware system simulated by software and running in a completely isolated environment.**
- **The JVM is an instruction set that simulates Java bytecode through software, and is the runtime environment for Java programs.**
- **Nowadays, the JVM not only supports Java language, but also Groovy, Kotlin and other languages can be converted into bytecode files, and the converted bytecode files can be run and processed through the Java virtual machine.**
- **properties:**
 - Compile once, run everywhere
 - Automatic memory management
 - Automatic garbage collection feature

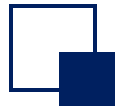


JVM architecture

❑ The JVM architecture consists of two main subsystems and two components:

- Class Loader and Execution Engine
- Runtime Data Area (JVM memory) and Native Interface





JVM garbage collection

- ❑ C and C++ use an explicit allocator to fully expose the heap space to the user, which has the advantage that skilled programmers can make good use of the heap space memory. The disadvantage is also obvious, it must be manually released after each allocation of memory, otherwise it is easy to cause memory leaks.



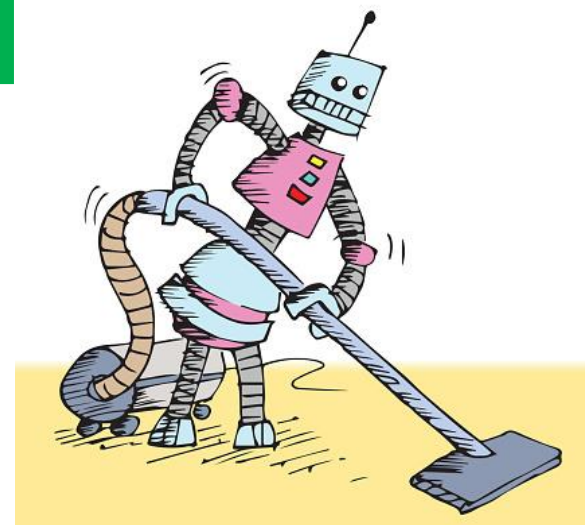
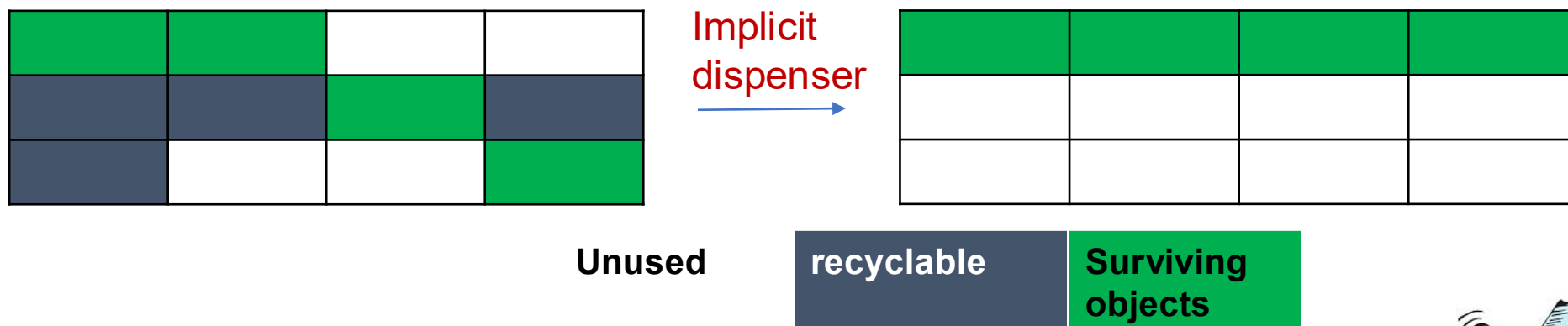
```
Void FunErrorA()  
{  
  String *p = new String[10];  
  // .....  
  delete p; // should use delete[] p;  
}
```

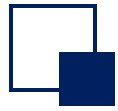
✗ memory leak



JVM garbage collection

- ❑ The core idea of Java is object-oriented, masking a lot of the low-level details and allowing programmers to focus more on "objects". Java uses an implicit allocator, where the programmer creates objects using heap memory and collects them to the garbage collector.
- ❑ Java's Garbage Collector In the execution engine, the main object of garbage collection is the JVM heap space.





JVM garbage collection

❑ Tasks of the garbage collector:

- Track and monitor every Java object, and reclaim the memory occupied by an object when it is in an unreachable state
- Clean up memory allocations and recycle memory fragments generated during the process.

❑ advantages:

- The details of memory management are shielded from developers, improving development efficiency.
- Developers do not have access to manipulate memory, reducing the risk of memory leaks.

❑ disadvantage:

- Garbage collection is not controlled by the developer, but is done by the JVM. In time-sensitive scenarios, uncontrolled garbage collection can incur excess time overhead.



The difference among JVM、JRE、JDK

- **JVM: Java Virtual Machine**

All JAVA Programs are running on JVM, JVM is part of JRE。

- **JRE: Java Runtime Environment**

JRE is mainly used for execute JAVA program, JRE includes JVM and some basic JAVA API, JRE is part of JDK.

- **JDK: Java Development Kit**

JDK provides JAVA development environment and runtime environment (JRE) .The development environment mainly includes some common tools, such as JAVA compiler, jar packagers, etc.





Chapter 2

- Identifiers, keywords, separators, comments
- Basic Java data types and constants
- Control process
- Inputs and outputs
- Array
- Exception mechanisms
- Java Virtual Machine and garbage collection